claude-windows / 73a3938e-1da3-4e28-bfa7-346a15718301

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d\agent-a8fc07a323b7c4b46.jsonl`
- SHA-256: `d46736a92463d5c55289ffbdfa942c4dd5effb5ab40ef802845f544ea5b1559d`
- Source modified: `2026-06-22T12:36:10+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_753ff171-b1d`
- session_id: `73a3938e-1da3-4e28-bfa7-346a15718301`

Transcript

1. user (2026-06-22T12:34:19.536Z)

You are fact-checking the DOC edit in PR #321 of the local git repo (Khelsutra/badminton-highlight-indexer) against the ACTUAL code/tests in PR #319. The working-agreement is strict: the doc must NOT assert any fabricated fact or invented provenance.

Read the doc edit: `git diff origin/master..origin/docs/data-gcs-train-blocker-resolved -- docs/DATA_IN_GCS.md`.
Read the actual code/tests it describes: `git show origin/fix/worker-train-dated-label-join:backend/worker/__main__.py` and `git show origin/fix/worker-train-dated-label-join:tests/test_worker.py`.

Check EVERY factual claim in the doc edit, e.g.:
- That the fix is the helper `_label_clip_name()` in `backend/worker/__main__.py` (it exists, named exactly that).
- That it strips both the `.rallies.csv`/`.csv` suffix AND a leading `^\d{4}-\d{2}-\d{2}_` prefix.
- That a dated label and a legacy bare label both resolve to the bare `videos/<name>` stem.
- That the claimed tests exist: "parametrized helper cases", "dated-label?bare-video cmd_train join regression", "per-label skip-branch coverage" ? verify each is a REAL test function in tests/test_worker.py.
- That the PR number/link (#319) is correct, and the §8 tracker row matches.
- That "synth train + all eval were unaffected" is true (they don't join videos).

Flag ANY claim that overstates, is unverifiable, or doesn't match the code. Return a verdict; ok=false for any inaccurate/fabricated claim.

2. assistant (2026-06-22T12:34:22.793Z)

I'll fact-check the doc edit in PR #321 against the actual code/tests in PR #319. Let me start by reading the relevant diffs and code.

3. user (2026-06-22T12:34:25.845Z)

=== FETCH DONE ===

4. assistant (2026-06-22T12:34:27.916Z)

Now let me read the doc diff and the actual code/tests in parallel.

5. user (2026-06-22T12:34:29.498Z)

diff --git a/docs/DATA_IN_GCS.md b/docs/DATA_IN_GCS.md
index 795e418..8581814 100644
--- a/docs/DATA_IN_GCS.md

```
+++ b/docs/DATA_IN_GCS.md
@@ -203,13 +203,16 @@ the bucket videos (don't trust the copied values ? a wrong fps silently
mis-al

 ## 7b. Known blockers ? must-fix before the DoD (2026-06-22 adversarial review)

-1. **Dated labels break `worker train --trajectory-source detector` (code-verified).**
`cmd_train`
-   derives the join key as `fname.replace(".rallies.csv","")` ? `2026-06-22_GX020094`, then
looks up
-   `video_by_stem.get(name)` against bare video stems (`GX020094`) ? so every clip is skipped
and train
-   aborts with `<2 videos` ([backend/worker/__main__.py:384](../backend/worker/__main__.py):384
vs
-   :378/:387). **Fix (recommended):** strip a leading `^\d{4}-\d{2}-\d{2}_` from the label-
derived
-   `name` in `cmd_train` (a ~1-line change, own PR + test) ? preserves the locked human-
readable naming
-   AND unblocks detector-train. *(`synth` train + all eval are unaffected ? they don't join
videos.)*
+1. **? RESOLVED 2026-06-22 ([#319](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/319))
+   ? dated labels broke `worker train --trajectory-source detector`.** *History:* `cmd_train`
derived the
+   join key as `fname.replace(".rallies.csv","")` ? `2026-06-22_GX020094`, then looked up
+   `video_by_stem.get(name)` against bare video stems (`GX020094`), so every clip was skipped
and train
+   aborted with `<2 videos`. **Fix:** the label?clip-name derivation is now the helper
`_label_clip_name()`
+   in [backend/worker/__main__.py](../backend/worker/__main__.py), which strips both the
+   `.rallies.csv`/`.csv` suffix AND a leading `^\d{4}-\d{2}-\d{2}_` prefix, so a dated label
and a legacy
+   bare label both resolve to the bare `videos/<name>` stem (preserves the locked human-
readable naming).
+   Tested: parametrized helper cases + a dated-label?bare-video `cmd_train` join regression +
per-label
+   skip-branch coverage. *(`synth` train + all eval were unaffected ? they don't join videos.)*
 2. **Phase-3 eval-on-GCS is net-new plumbing, not a flag** ? see §6 Phase-3. Eval is local-FS
only.
 3. **`manifests/golden.json` is consumed by zero code today** (train ignores it; eval can't
fetch
    `gs://`) ? it's correct-but-dormant until blocker #2 lands. Add a tiny manifest-validator +
a CI job
@@ -233,6 +236,7 @@ the bucket videos (don't trust the copied values ? a wrong fps silently mis-
al
 | Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
 | Phase 1: eval manifest ? `manifests/golden.json` | ? 2026-06-22 |
 | Phase 1: remove 7 stale `_proxy`/bare label objects | ? 2026-06-22 (MD5-verified dupes,
deleted) |
+| Blocker §7b#1: dated labels join in `worker train --trajectory-source detector` | ?
2026-06-22 ([#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319)) ?
`_label_clip_name` strips the ISO-date prefix |
 | Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done; **1/15**
demo'd (`mahadevpura_1`); 14 remain ? run `rclone` on a cloud box (preferred) or the owner-box
script |
 | Phase 2: proxy the 8 large F: full-MP4 clips before upload | ? |
 | Phase 3: eval/litmus default to the GCS manifest | ? |
```

6. user (2026-06-22T12:34:30.072Z)

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
```

```
The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""
from __future__ import annotations

import argparse
import csv
import json
import logging
import os
import re
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.compute import ComputeJobSpec, get_compute_backend
from backend.config import (
    StartupCheckError,
    enforce_startup_checks,
    load_config,
    probe_cuda_available,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
from backend.utils.hashing import compute_video_id

logger = logging.getLogger("backend.worker")


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
        try:
            return cast(v)
        except ValueError:
            pass
    return v


def _apply_overrides(config: Any, sets: List[str]) -> None:
    """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
    for kv in sets or []:
        key, sep, val = kv.partition("=")
        if not sep:
            raise ValueError(f"--set expects k=v, got {kv!r}")
        parts = key.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p)
        setattr(obj, parts[-1], _coerce(val))
```

```
def _write_intervals_csv(segments: List[dict], path: str) -> None:
    """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-friendly
    artifact (same schema as the rally-annotator labels)."""
    with open(path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["start", "end", "shot_count", "ending_reason"])
        for s in segments:
            w.writerow([
                f'{float(s.get("start_time", 0.0)):.3f}',
                f'{float(s.get("end_time", 0.0)):.3f}',
                s.get("shot_count", ""),
                s.get("ending_reason", s.get("shot_count_confidence", "")),
            ])


def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) -> None:
    with open(path, "w") as f:
        json.dump({
            "video_id": video_id,
            "status": status,
            "segment_count": len(segments),
            "segments": [{"start": float(s.get("start_time", 0.0)),
                          "end": float(s.get("end_time", 0.0))} for s in segments],
        }, f, indent=2)


def _run_startup_checks(config: Any) -> bool:
    """M5 per-profile startup checks for the worker (the compute process, so it probes CUDA
    best-effort). Logs warnings; returns False on a FATAL coherence error so the caller aborts
    cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true no-op
    (returns True, ZERO work) when no deployment.profile is selected.

    The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily imports
    torch, and pre-M5 the worker's infer/train path was torch-free (detection is delegated to a
    WSL/native subprocess). Probing only when a profile is set keeps the default-OFF path a true
    no-op of work, not just of output."""
    profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
    cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF path
    try:
        enforce_startup_checks(config, cuda_available=cuda, logger=logger)
        return True
    except StartupCheckError:
        return False  # already logged at ERROR by enforce_startup_checks


def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc once the
    outputs are durably in the bucket (the backend bucket-polls the done-marker). The dispatched
    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.

    A remote job that never writes a completion marker (a failed/hung container) makes the
bucket
    poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
dispatch
    timeout/failure, distinct from the local 2/3) rather than letting a raw traceback escape."""
    from backend.infrastructure.execution_policy import PolicyTimeout

    spec = ComputeJobSpec(
        video_uri=args.video_uri, out_uri=args.out_uri,
        segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
    )
    try:
        result = compute.run_infer(spec)
    except PolicyTimeout as e:
```

```
        logger.warning("[worker] remote compute did not complete (no marker within budget): %s",
e)
        return 4
    logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
                result.video_id, result.returncode, result.outputs_uri)
    return result.returncode


def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count: int,
                       status: str, storage_cfg: dict, scratch: str) -> None:
    """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a
remote
    dispatcher's bucket-poll. Best-effort ? never raises into the worker's success return."""
    try:
        marker = {"video_id": video_id, "returncode": returncode,
                  "segment_count": segment_count, "status": status}
        local = os.path.join(scratch, "_done.json")
        with open(local, "w", encoding="utf-8") as f:
            json.dump(marker, f)
        storage.put(local, done_uri, config=storage_cfg)
        logger.info("[worker] wrote completion marker -> %s", done_uri)
    except Exception as e:  # never fail a good run because the marker push hiccuped
        logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


def cmd_infer(args: argparse.Namespace) -> int:
    config = load_config(args.config) if args.config else load_config()
    _apply_overrides(config, args.set)
    if not _run_startup_checks(config):
        return 2

    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-process.
    # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall through to
the
    # unchanged inline path below. The dispatched container runs THIS cmd_infer inline (the
    # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.
    compute = get_compute_backend(config)
    if compute is not None:
        return _dispatch_remote(compute, args)

    storage_cfg = config.storage.model_dump()

    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
    os.makedirs(scratch, exist_ok=True)
    config.output.output_dir = scratch

    # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to
scratch.
    local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
    print(f"[worker] pulled {args.video_uri} -> {local_video}")

    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

    # 3. VALIDATE (same registry as the main CLI).
    val_results, val_context = validator_registry.run_all_with_context(local_video, config)
    failures = [r for r in val_results if not r.passed]
    db = Database(os.path.join(scratch, config.output.db_name))
    db.add_video(video_id, local_video, "failed" if failures else "processed",
                 [r.model_dump() for r in val_results])

    segments: List[dict] = []
    segmenter_actual = None
```

```
    segmentation_ran = False
    status = "failed"
    try:
        if failures:
            print("[worker] validation FAILED: "
                  + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
            return 2
        seg_name = args.segmenter or config.indexing.default_segmenter
        segmenter_cls = segmenter_registry.get(seg_name)
        if not segmenter_cls:
            print(f"[worker] unknown segmenter: {seg_name!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return 2
        segmenter_actual = seg_name
        result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
        segmentation_ran = True
        if isinstance(result, Failure):
            print(f"[worker] segmenter FAILED: {result.message}")
            return 3
        segments = result.value if hasattr(result, "value") else result
        status = "success"
        # Loud, never-silent diagnostic: a 0-segment success is almost always a misconfig, not
an
        # empty match ? the cold-start failure mode (the substrate detector yielded no usable
        # trajectories). Explain it so a run is analysable from the log alone (re-run -v for
more).
        if not segments:
            detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
                             or getattr(config.indexing, "detector_impl", None) or "auto")
            logger.warning(
                "infer produced 0 segments for video_id=%s (segmenter=%s, detector_impl=%s). "
                "The detector substrate yielded no usable trajectories/windows ? check the
detector "
                "logs above; common causes: native runner UNAVAILABLE (weights/repo/WASB_PYTHON
env), "
                "the WASB env produced no detections, or the input resolution differs from the
tuned "
                "proxy resolution. Re-run with -v for the native command + frame counts.",
                video_id, segmenter_actual, detector_impl,
            )
    finally:
        # Best-effort per-video observability report (never raises) ? parity with the main CLI.
        emit_indexer_report(
            db=db, video_id=video_id, video_path=local_video, config=config,
            val_results=val_results, val_context=val_context,
            segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
            was_reingest=False, segmentation_ran=segmentation_ran,
        )

    # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
    out_local = os.path.join(scratch, "outputs", video_id)
    os.makedirs(out_local, exist_ok=True)
    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
    _write_report_json(video_id, segments, status, os.path.join(out_local, "report.json"))

    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(out_local)):
        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
        pushed.append(uri)

    print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)} artifact(s):")
    for u in pushed:
```

```
        print("   ", u)

    # M6: a remote dispatcher passes --done-uri; write a completion marker so its bucket-poll
    # terminates (carries video_id ? the dispatcher's output path + the worker rc). DEFAULT-OFF:
    # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a failed
container
    # job currently lets the dispatcher's bounded poll time out (a failure-marker is a follow-
up).
    if getattr(args, "done_uri", None):
        _write_done_marker(args.done_uri, video_id, 0, len(segments), status, storage_cfg,
scratch)
    return 0


#: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter so a
#: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
_VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

#: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
#: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date but is NOT
#: part of the clip's join key, so it's stripped when deriving the videos/ stem.
_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


def _label_clip_name(fname: str) -> str:
    """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label filename.

    Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date prefix,
so a
    canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

    * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-stamped)
    * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same stem)
    * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of the name,
kept)
    * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in <name>
survive)

    Without the date strip, ``--trajectory-source detector`` can't match a dated label to its
bare
    video stem, so every clip is skipped and train aborts with ``<2 videos`` (DATA_IN_GCS §7b
#1).
    """
    name = fname.replace(".rallies.csv", "").replace(".csv", "")
    return _LABEL_DATE_PREFIX.sub("", name)


def _probe_video_fps_width(path: str):
    """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

    The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a WRONG
    fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles every time).
    cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for the
    inference report (it flags the fallback) but corrupting for REAL training data. So here we
    probe with ffprobe (same tool as ``get_video_duration``) and return None so the caller SKIPS
    rather than guesses.
    """
    import json
    import subprocess
    try:
        out = subprocess.check_output(
            [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
             "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
            stderr=subprocess.DEVNULL).decode()
```

```
        st = (json.loads(out).get("streams") or [{}])[0]
        num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
        fps = float(num) / float(den or "1")
        width = float(st.get("width") or 0)
        if fps > 0 and width > 0:
            return fps, width
    except (subprocess.SubprocessError, ValueError, KeyError, OSError, json.JSONDecodeError):
        pass
    return None


def _resolve_train_detector(args: argparse.Namespace, config: Any):
    """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
    detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU box,
    stub=CPU mock). Returns the runner, or prints an error + returns None.

    Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build the
    detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo) has
    no shuttle detector and is rejected.
    """
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

    seg_cls = segmenter_registry.get(args.segmenter)
    if not seg_cls:
        print(f"[worker] unknown segmenter: {args.segmenter!r} "
              f"(have {list(segmenter_registry.list_available())})")
        return None
    seg = seg_cls(None, config)
    if not isinstance(seg, TrajectoryHybridSegmenter):
        print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
              f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid).")
        return None
    try:
        runner, _ = seg._resolve_runner(config.get("indexing", {}))
    except NotImplementedError as e:
        # e.g. detector_impl='native' for a family without a native runner (tracknet, or
        # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw traceback.
        print(f"[worker] detector not available: {e}")
        return None
    ok, msg = runner.healthcheck()
    if not ok:
        print(f"[worker] detector environment not ready: {msg}")
        return None
    print(f"[worker] detector ready ({args.segmenter}): {msg}")
    return runner


def cmd_train(args: argparse.Namespace) -> int:
    """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest -> shell out
    to training.gen0.harness (backend must NOT import training) -> push the artifact bundle.

    Two trajectory sources (the train pipeline + harness are identical either way):
      * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent synthetic
        shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box / CI.
      * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
        DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test). This
        is the M3.5 "first real training" path; only the trajectory source flips.
    """
    import subprocess

    config = load_config(args.config) if args.config else load_config()
    _apply_overrides(config, args.set)
    if not _run_startup_checks(config):
        return 2
    storage_cfg = config.storage.model_dump()
```

```
    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
    labels_dir = os.path.join(scratch, "labels")
    traj_dir = os.path.join(scratch, "trajectories")
    videos_dir = os.path.join(scratch, "videos")
    os.makedirs(labels_dir, exist_ok=True)
    os.makedirs(traj_dir, exist_ok=True)

    corpus = args.corpus_uri.rstrip("/")
    label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
                        if u.lower().endswith(".csv"))
    if len(label_uris) < 2:
        print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
              f"under {corpus}/labels/")
        return 2

    # Real-detector source: resolve the runner once + index the bucket's videos/ by stem.
    runner = None
    video_by_stem: dict = {}
    if args.trajectory_source == "detector":
        os.makedirs(videos_dir, exist_ok=True)
        runner = _resolve_train_detector(args, config)
        if runner is None:
            return 2
        for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
            vname = storage.parse_uri(vuri).name
            if not vname.lower().endswith(_VIDEO_EXTS):
                continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow the video
            video_by_stem[os.path.splitext(vname)[0]] = vuri

    print(f"[worker] trajectory source: {args.trajectory_source}")
    specs: List[dict] = []
    for uri in label_uris:
        fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
        name = _label_clip_name(fname)                         # -> GX020094 (date prefix
stripped)
        local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
        if args.trajectory_source == "detector":
            vuri = video_by_stem.get(name)
            if not vuri:
                print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
                continue
            local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                        config=storage_cfg)
            video_id = compute_video_id(local_video)
            # Real fps/width drive the trajectory frame->time mapping; PROBE (don't guess) ? a
            # wrong fps silently mis-aligns every label, so skip the video if ffprobe can't read
it.
            meta = _probe_video_fps_width(local_video)
            if meta is None:
                print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ? skipping
(won't guess).")
                continue
            fps, frame_width = meta
            traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
            if not traj:
                print(f"[worker] detector produced no trajectory for {name!r} ? skipping.")
                continue
            specs.append({"name": name, "trajectory": traj, "labels": local_label,
                          "fps": fps, "frame_width": frame_width})
        else:
            from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
            traj = os.path.join(traj_dir, f"{name}_traj.csv")
```

```
            synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
            specs.append({"name": name, "trajectory": traj, "labels": local_label,
                          "fps": args.fps, "frame_width": args.frame_width})

    if len(specs) < 2:
        print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
        return 2
    manifest_path = os.path.join(scratch, "manifest.json")
    with open(manifest_path, "w", encoding="utf-8") as f:
        json.dump(specs, f, indent=2)
    src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
    print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

    out_dir = os.path.join(scratch, "artifacts")
    cmd = [sys.executable, "-m", "training.gen0.harness",
           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
    if args.floor:
        floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                       if "://" in args.floor else args.floor)
        cmd += ["--floor", floor_local]
    print("[worker] training:", " ".join(cmd))
    rc = subprocess.run(cmd).returncode
    if rc != 0:
        print(f"[worker] gen0 harness failed (rc={rc})")
        return rc

    # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
    bundle = os.path.join(out_dir, args.version)
    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(bundle)):
        uri = f"{out_prefix}/weights/{args.version}/{fn}"
        storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
        pushed.append(uri)
    print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("   ", u)
    return 0


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="python -m backend.worker",
                                description="Storage-aware worker: pull ? run pipeline ? push.")
    p.add_argument("-v", "--verbose", action="store_true",
                   help="DEBUG logging (the native detector command, frame counts, per-stage
detail)")
    sub = p.add_subparsers(dest="cmd", required=True)

    pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
    pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// / gcs://")
    pi.add_argument("--out-uri", required=True, help="output prefix (outputs/<video_id>/? is
appended)")
    pi.add_argument("--segmenter", default=None, help="default: indexing.default_segmenter")
    pi.add_argument("--config", default=None, help="config.json path (default: ./config.json)")
    pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp dir)")
    pi.add_argument("--max-frames", type=int, default=None)
    pi.add_argument("--done-uri", default=None,
                    help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
                         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll
terminates. "
                         "Default-OFF: omit it for the normal local run.")
    pi.add_argument("--set", action="append", default=[], metavar="k=v",
```

```
                    help="config override, e.g. --set indexing.skip_ai_handoff=true")
    pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] -> trajectories
-> harness)")
    pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+ videos/
for "
                    "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
    pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/ is
appended)")
    pt.add_argument("--version", required=True, help="semver for the artifact bundle")
    pt.add_argument("--trajectory-source", choices=["synth", "detector"], default="synth",
                    help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
                         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
                         "(needs videos/ in the corpus; honours detector_impl).")
    pt.add_argument("--segmenter", default="wasb_hybrid",
                    help="trajectory hybrid whose detector is used for --trajectory-source
detector")
    pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or storage
URI)")
    pt.add_argument("--config", default=None)
    pt.add_argument("--scratch", default=None)
    pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps (detector
probes the video)")
    pt.add_argument("--frame-width", type=float, default=1280.0,
                    help="synth trajectory frame width (detector probes the video)")
    pt.add_argument("--set", action="append", default=[], metavar="k=v")
    pt.set_defaults(func=cmd_train)
    return p


def _setup_logging(verbose: bool) -> None:
    """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR diagnostics
are
    VISIBLE ? the worker previously ran with no logging config, so every segmenter/detector log
    (detector choice, 'Running native WASB inference', availability failures) was silently
dropped.
    That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here (the CLI
    entry), not at import, so importing the module never reconfigures a caller's logging."""
    # force=True: basicConfig is a no-op if the root already has handlers (e.g. a library
    # configured logging at import), which would leave the level at WARNING and re-suppress the
    # diagnostics. Forcing reconfiguration means the worker reliably controls its own verbosity.
    logging.basicConfig(
        level=logging.DEBUG if verbose else logging.INFO,
        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
        datefmt="%H:%M:%S",
        force=True,
    )


def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    _setup_logging(getattr(args, "verbose", False))
    return args.func(args)


if __name__ == "__main__":
    sys.exit(main())
```

7. assistant (2026-06-22T12:34:35.589Z)

The code confirms `_label_clip_name()` exists with the described behavior. Now let me read the tests in PR #319.

8. user (2026-06-22T12:34:36.570Z)

```
"""Worker dispatcher: pull (storage) ? run pipeline ? push (storage).

`cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is validated
on the droplet against MinIO.
"""
import json

import pytest

from backend.config import load_config
from backend.worker import __main__ as wmain


# ---------------------------------------------------------- pure helpers


@pytest.mark.parametrize("raw,expected", [
    ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid", "wasb_hybrid"),
])
def test_coerce(raw, expected):
    assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


def test_apply_overrides_on_typed_config():
    cfg = _fresh_config()
    wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
                                 "sport=tennis"])
    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.min_segment_duration == 1.5
    assert cfg.sport == "tennis"


def test_apply_overrides_rejects_bad_format():
    cfg = _fresh_config()
    with pytest.raises(ValueError):
        wmain._apply_overrides(cfg, ["no_equals_sign"])


def test_write_intervals_csv(tmp_path):
    segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4, "shot_count_confidence":
"LocalNoAI"},
            {"start_time": 9.0, "end_time": 12.5}]
    out = tmp_path / "intervals.csv"
    wmain._write_intervals_csv(segs, str(out))
    lines = out.read_text().splitlines()
    assert lines[0] == "start,end,shot_count,ending_reason"
    assert lines[1].startswith("2.000,5.000,4,")
    assert lines[2].startswith("9.000,12.500,,")


def test_write_report_json(tmp_path):
    out = tmp_path / "report.json"
    wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
str(out))
    data = json.loads(out.read_text())
    assert data["video_id"] == "vid1" and data["segment_count"] == 1
    assert data["segments"][0] == {"start": 1.0, "end": 2.0}


def test_parser_infer_accepts_set_and_uris():
    args = wmain.build_parser().parse_args([
        "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
        "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
```

```
    assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]


## ------------------------------------------------------------ cmd_infer (local, mocked)

class _OKResult:
    passed = True
    validator_name = "fake"
    message = "ok"

    def model_dump(self):
        return {"validator_name": "fake", "passed": True, "message": "ok"}


class _FailResult(_OKResult):
    passed = False
    message = "too blurry"


class _FakeSeg:
    def __init__(self, db, config):
        pass

    def process_video(self, video_id, path, max_frames=None):
        return [
            {"start_time": 2.0, "end_time": 5.0, "shot_count": 4, "shot_count_confidence":
"LocalNoAI"},
            {"start_time": 9.0, "end_time": 12.0, "shot_count": 6, "shot_count_confidence":
"LocalNoAI"},
        ]


def _fresh_config():
    # Load defaults without touching any cwd config.json.
    import tempfile
    p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
    p.write("{}")
    p.close()
    return load_config(p.name)


@pytest.fixture
def mocked_pipeline(monkeypatch):
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                        lambda path, cfg: ([_OKResult()], {}))
    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)


def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
    video = tmp_path / "in.mp4"
    video.write_bytes(b"FAKEVIDEO")
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")
    out = tmp_path / "bucket"
    scratch = tmp_path / "scratch"

    rc = wmain.main([
        "infer", "--video-uri", str(video), "--out-uri", str(out),
        "--config", str(cfg), "--scratch", str(scratch),
        "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
    ])
```

```
    assert rc == 0
    intervals = out / "outputs" / "vid123" / "intervals.csv"
    report = out / "outputs" / "vid123" / "report.json"
    assert intervals.exists() and report.exists()
    rows = intervals.read_text().splitlines()
    assert len(rows) == 3                          # header + 2 rallies
    assert json.loads(report.read_text())["segment_count"] == 2


class _EmptySeg:
    """A segmenter that finds nothing ? the cold-start failure mode."""

    def __init__(self, db, config):
        pass

    def process_video(self, video_id, path, max_frames=None):
        return []


def test_setup_logging_levels():
    import logging
    wmain._setup_logging(False)
    assert logging.getLogger().level == logging.INFO
    wmain._setup_logging(True)
    assert logging.getLogger().level == logging.DEBUG


def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
    # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start bug).
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                        lambda path, cfg: ([_OKResult()], {}))
    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
    video = tmp_path / "in.mp4"
    video.write_bytes(b"FAKE")
    out = tmp_path / "bucket"

    # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures handlers
and
    # would detach caplog's). The warning is emitted by cmd_infer regardless of who configures
logging.
    args = wmain.build_parser().parse_args([
        "infer", "--video-uri", str(video), "--out-uri", str(out),
        "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"])
    with caplog.at_level("WARNING", logger="backend.worker"):
        rc = wmain.cmd_infer(args)
    assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
    msgs = " ".join(r.getMessage() for r in caplog.records)
    assert "0 segments" in msgs and "vidZ" in msgs
    assert "detector_impl=native" in msgs       # surfaces the resolved detector
    assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
    assert json.loads((out / "outputs" / "vidZ" / "report.json").read_text())["segment_count"]
== 0


def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                        lambda path, cfg: ([_FailResult()], {}))
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
    video = tmp_path / "in.mp4"
    video.write_bytes(b"X")
    cfg = tmp_path / "config.json"
```

```
    cfg.write_text("{}")
    rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "o"),
                     "--config", str(cfg), "--scratch", str(tmp_path / "s")])
    assert rc == 2
```

--- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

```
@pytest.mark.parametrize("fname,expected", [
    ("2026-06-22_GX020094.rallies.csv", "GX020094"),           # canonical dated label
(DATA_IN_GCS §7.1)
    ("GX020094.rallies.csv", "GX020094"),                      # legacy bare label -> SAME clip
stem
    ("GX020094_proxy.rallies.csv", "GX020094_proxy"),          # _proxy is part of the name,
kept
    ("2026-06-21_mahadevpura_1.rallies.csv", "mahadevpura_1"),  # underscores in <name> survive
    ("v1.csv", "v1"),                                          # bare .csv suffix
    ("2026-06-22_GX020094.csv", "GX020094"),                   # dated + bare .csv
])
def test_label_clip_name_strips_date_prefix(fname, expected):
    """A dated label and a bare label must both resolve to the bare videos/<name> stem."""
    assert wmain._label_clip_name(fname) == expected


def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
    """Regression (DATA_IN_GCS §7b blocker #1): canonical labels carry an ISO-date prefix
    (labels/<date>_<name>.rallies.csv) while videos are bare (videos/<name>.mp4). The label-
derived
    clip name must strip the date so the video join succeeds ? otherwise every clip is skipped
and
    train aborts with <2 videos."""
    import os
    import subprocess

    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # CPU stub runner (no GPU/WSL)
    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    videos_dir = corpus / "videos"
    labels_dir.mkdir(parents=True)
    videos_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    # canonical DATED labels ...
    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n")
    (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header +
"1,3.0,6.0,winner,badminton,\n")
    # ... joining to BARE-stem videos
    (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
    (videos_dir / "mahadevpura_1.mp4").write_bytes(b"FAKEVIDEO2")
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
```

```
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                     "--version", "0.0.4", "--config", str(cfg), "--scratch", str(scratch),
                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
    assert rc == 0  # before the fix: every clip skipped -> rc 2 ("<2 videos")
    specs = json.loads((scratch / "manifest.json").read_text())
    assert len(specs) == 2                                      # both dated labels joined
    assert {s["name"] for s in specs} == {"GX020094", "mahadevpura_1"}  # downstream name is
date-stripped


def test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
    """A dated label whose <name> has no videos/<name>.<ext> is SKIPPED (the join-miss branch),
    while the labels that DO join are kept ? so one orphan label can't abort an otherwise-valid
run."""
    import os
    import subprocess

    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    videos_dir = corpus / "videos"
    labels_dir.mkdir(parents=True)
    videos_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    row = "1,2.0,5.0,winner,badminton,\n"
    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header + row)
    (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header + row)
    (labels_dir / "2026-06-20_orphan.rallies.csv").write_text(header + row)  # NO
videos/orphan.*
    (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
    (videos_dir / "mahadevpura_1.mp4").write_bytes(b"FAKEVIDEO2")
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                     "--version", "0.0.5", "--config", str(cfg), "--scratch", str(scratch),
                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
    assert rc == 0  # the orphan label was skipped; the two joinable clips carried the run
    specs = json.loads((scratch / "manifest.json").read_text())
    assert {s["name"] for s in specs} == {"GX020094", "mahadevpura_1"}  # 'orphan' absent


def test_cmd_train_detector_skips_unprobeable_and_trajless_videos(tmp_path, monkeypatch):
    """cmd_train's detector loop must SKIP (never crash, never guess) a video whose fps/width
won't
    probe and a video the detector yields no trajectory for, keeping the healthy clips."""
    import os
    import subprocess
```

```
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    videos_dir = corpus / "videos"
    labels_dir.mkdir(parents=True)
    videos_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    row = "1,2.0,5.0,winner,badminton,\n"
    for stem in ("good1", "good2", "badfps", "notraj"):
        (labels_dir / f"2026-06-22_{stem}.rallies.csv").write_text(header + row)
        (videos_dir / f"{stem}.mp4").write_bytes(b"FAKE-" + stem.encode())

    # probe fails ONLY for badfps -> that clip is skipped (a wrong fps mis-aligns every label).
    monkeypatch.setattr(wmain, "_probe_video_fps_width",
                        lambda p: None if "badfps" in os.path.basename(p) else (30.0, 1280.0))
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "id-" + os.path.basename(p))

    class _FakeRunner:
        # writes a real trajectory for everything EXCEPT notraj (empty result -> skipped).
        def run_predict(self, video, out_dir, video_id=None):
            if "notraj" in os.path.basename(video):
                return ""
            traj = os.path.join(out_dir, f"{video_id}_traj.csv")
            with open(traj, "w") as f:
                f.write("frame,x,y\n0,1,2\n")
            return traj
    monkeypatch.setattr(wmain, "_resolve_train_detector", lambda args, config: _FakeRunner())

    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                     "--version", "0.0.6", "--config", str(cfg), "--scratch", str(scratch),
                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
    assert rc == 0
    specs = json.loads((scratch / "manifest.json").read_text())
    assert {s["name"] for s in specs} == {"good1", "good2"}  # badfps + notraj skipped


def test_cmd_train_detector_aborts_clean_when_fewer_than_two_videos_join(tmp_path, monkeypatch):
    """When labels join to <2 videos, train aborts cleanly with rc 2 (leave-one-video-out needs
>=2).
    This is the guard the date-prefix bug used to trip for EVERY clip (DATA_IN_GCS §7b #1); here
it
    fires legitimately because only one of two labels has a matching video."""
    import subprocess

    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    videos_dir = corpus / "videos"
    labels_dir.mkdir(parents=True)
```

```
    videos_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    row = "1,2.0,5.0,winner,badminton,\n"
    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header + row)  # joins
    (labels_dir / "2026-06-21_orphan.rallies.csv").write_text(header + row)    # no video ->
skipped
    (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")
    # subprocess.run must NOT be reached (we abort before the harness); guard that.
    monkeypatch.setattr(subprocess, "run", lambda *a, **k: pytest.fail("harness ran despite <2
videos"))

    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(tmp_path / "out"),
                     "--version", "0.0.7", "--config", str(cfg), "--scratch", str(tmp_path /
"s"),
                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
    assert rc == 2  # only 1 of 2 labels joined -> below the leave-one-video-out floor


def test_cmd_train_orchestration(tmp_path, monkeypatch):
    import os
    import subprocess

    labels_dir = tmp_path / "corpus" / "labels"
    labels_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    (labels_dir / "v1.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
    (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
str(out_dir),
                     "--version", "0.0.1", "--config", str(cfg), "--scratch", str(scratch)])
    assert rc == 0
    # bundle pushed to <out-uri>/weights/<version>/
    assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
    assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
    # the manifest fed to the harness enumerated both labeled videos with a trajectory each
    specs = json.loads((scratch / "manifest.json").read_text())
    assert len(specs) == 2
    assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)


def test_parser_train_detector_args():
    args = wmain.build_parser().parse_args([
        "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version", "1.0.0",
        "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
    assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
```

```
def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
    """--trajectory-source detector pulls videos + runs the resolved DetectorRunner. With
    RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the runner)."""
    import os
    import subprocess

    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no GPU/WSL)
    # Fake bytes aren't decodable, so stub the (strict, fail-loud) ffprobe probe to a real
value.
    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    videos_dir = corpus / "videos"
    labels_dir.mkdir(parents=True)
    videos_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
    (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
    (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
    (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
    (videos_dir / "v1.srt").write_text("1\n00:00:00,000 --> 00:00:01,000\nx\n")  # sidecar: must
NOT shadow v1.mp4
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                     "--version", "0.0.2", "--config", str(cfg), "--scratch", str(scratch),
                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
    assert rc == 0
    specs = json.loads((scratch / "manifest.json").read_text())
    assert len(specs) == 2  # the v1.srt sidecar did NOT shadow v1.mp4 (extension filter)
    # Real-extract path: each spec points at an on-disk trajectory the runner actually wrote.
    assert all(os.path.exists(s["trajectory"]) for s in specs)
    assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub runner's output


def test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(tmp_path, monkeypatch):
    """detector_impl=native for a family with no native runner must return rc 2 (clean), not
    propagate a raw NotImplementedError traceback out of the worker."""
    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    (labels_dir).mkdir(parents=True)
    (corpus / "videos").mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
    (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")
    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(tmp_path / "out"),
                     "--version", "0.0.3", "--config", str(cfg), "--scratch", str(tmp_path /
"s"),
```

```
                     "--trajectory-source", "detector", "--segmenter", "tracknet_hybrid"])
    assert rc == 2  # graceful: _resolve_train_detector caught NotImplementedError, returned
None


def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
    """The strict ffprobe probe must return None (caller skips) for a non-video file, never
guess."""
    bad = tmp_path / "not_a_video.mp4"
    bad.write_bytes(b"NOTAVIDEO")
    assert wmain._probe_video_fps_width(str(bad)) is None
```

9. assistant (2026-06-22T12:34:40.302Z)

I now have both the code and tests. Let me verify the PR number/branch correspondence and check the §8 tracker context. Let me confirm which PR number corresponds to the `fix/worker-train-dated-label-join` branch and verify the doc's claims about PR #319.

10. user (2026-06-22T12:34:43.224Z)

```
{"headRefName":"fix/worker-train-dated-label-
join","number":319,"state":"OPEN","title":"fix(worker): strip ISO-date prefix when joining
labels to videos in train","url":"https://github.com/Khelsutra/badminton-highlight-
indexer/pull/319"}
=== 321 ===
{"headRefName":"docs/data-gcs-train-blocker-
resolved","number":321,"state":"OPEN","title":"docs(data): mark DATA_IN_GCS §7b blocker #1
resolved by #319","url":"https://github.com/Khelsutra/badminton-highlight-indexer/pull/321"}
```

11. assistant (2026-06-22T12:34:47.612Z)

PR #319 is the `fix/worker-train-dated-label-join` branch, and PR #321 is the doc branch. The PR number/link mapping is correct. Now let me verify the doc's reference to "§8 tracker row" ? the doc says "§8 tracker row matches". Let me check what section the tracker is in, since the prompt mentions §8 but the diff shows the table row.

12. user (2026-06-22T12:34:49.116Z)

```
1:# Data in GCS ? the corpus source-of-truth map
20:## 1. The bucket
35:## 2. Canonical layout (the target contract)
60:## 3. What's actually in the bucket today (2026-06-22)
65:> (14 of 15) ? see §6/§7b/§8.
83:## 4. How the cloud consumes it
89:# config (no secrets in config; auth via GOOGLE_APPLICATION_CREDENTIALS):
90:#   { "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826" } } }
92:# INFERENCE on one cloud video -> outputs/<video_id>/ in the bucket
97:# TRAIN from the bucket (reads labels/ [+ videos/], extracts trajectories, builds manifest
on-box,
98:# runs training.gen0.harness, pushes weights/<version>/):
109:## 5. The gap ? what's still local-only (why the box is still a blocker)
128:## 6. Migration plan (close the gap)
135:# trajectories (decision-free; absent today)
137:# labels (after the §7.1 naming decision) -> labels/<canonical-name>.rallies.csv
138:# eval manifest with gs:// (or bucket-relative) paths -> a stable manifest object (§7.3)
161:(same as the label cleanup). Track per-clip in §8.
174:## 7. Decisions
204:## 7b. Known blockers ? must-fix before the DoD (2026-06-22 adversarial review)
228:## 8. Progress tracker
239:| Blocker §7b#1: dated labels join in `worker train --trajectory-source detector` | ?
2026-06-22 ([#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319)) ?
`_label_clip_name` strips the ISO-date prefix |
249:## Related
```

13. assistant (2026-06-22T12:34:56.679Z)

The §8 tracker row is confirmed (line 239, under "## 8. Progress tracker"). Now let me verify each specific claim against what I've read. Let me check the key claims one by one.

Let me verify the claim about the §8.1 naming convention reference and the helper's exact behavior with the `.csv` ordering. The code does `fname.replace(".rallies.csv","").replace(".csv","")` then strips the date prefix. Let me verify the test function names exist precisely as the doc describes them.

The doc claims three tested categories:
1. "parametrized helper cases" ? `test_label_clip_name_strips_date_prefix` (parametrized, 6 cases) ?
2. "a dated-label?bare-video `cmd_train` join regression" ? `test_cmd_train_detector_joins_dated_labels_to_bare_videos` ?
3. "per-label skip-branch coverage" ? `test_cmd_train_detector_skips_label_with_no_matching_video` ?

Let me verify the "synth train + all eval were unaffected" claim by checking that synth path doesn't use `video_by_stem` join, and confirm the suffix-stripping claim about both `.rallies.csv` and `.csv`.

14. user (2026-06-22T12:34:59.364Z)

```
=== synth branch: does it use video_by_stem join? ===
392:    video_by_stem: dict = {}
393:    if args.trajectory_source == "detector":
402:            video_by_stem[os.path.splitext(vname)[0]] = vuri
410:        if args.trajectory_source == "detector":
411:            vuri = video_by_stem.get(name)
431:        else:
432:            from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
434:            synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
444:    src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"

=== verify _label_clip_name still called for synth (same name var) ===
274:def _label_clip_name(fname: str) -> str:
408:        name = _label_clip_name(fname)                        # -> GX020094 (date prefix
stripped)
```

15. assistant (2026-06-22T12:35:04.076Z)

Confirmed. The synth branch (`else:` at line 431) does NOT touch `video_by_stem` ? it only uses `name` for the trajectory filename and synth generation, not for any video join. So "synth train was unaffected" is accurate. Now let me verify the "all eval" claim ? eval is a separate code path. The doc's broader claim is that eval doesn't join videos. Let me confirm eval isn't in this worker file (it's local-FS only per blocker #2) and verify the master version of the doc to ensure the "history" recounting is faithful.

16. user (2026-06-22T12:35:05.401Z)

```
=== master version of the blocker #1 text (what the doc says was the OLD behavior) ===
```

7b. Known blockers ? must-fix before the DoD (2026-06-22 adversarial review)

```
1. **Dated labels break `worker train --trajectory-source detector` (code-verified).**
`cmd_train`
   derives the join key as `fname.replace(".rallies.csv","")` ? `2026-06-22_GX020094`, then
looks up
   `video_by_stem.get(name)` against bare video stems (`GX020094`) ? so every clip is skipped
and train
   aborts with `<2 videos` ([backend/worker/__main__.py:384](../backend/worker/__main__.py):384
vs
```

   :378/:387). **Fix (recommended):** strip a leading `^\d{4}-\d{2}-\d{2}_` from the label-
derived
   `name` in `cmd_train` (a ~1-line change, own PR + test) ? preserves the locked human-readable
naming
   AND unblocks detector-train. *(`synth` train + all eval are unaffected ? they don't join
videos.)*
2. **Phase-3 eval-on-GCS is net-new plumbing, not a flag** ? see §6 Phase-3. Eval is local-FS
only.
3. **`manifests/golden.json` is consumed by zero code today** (train ignores it; eval can't
fetch
   `gs://`) ? it's correct-but-dormant until blocker #2 lands. Add a tiny manifest-validator + a
CI job

17. assistant (2026-06-22T12:35:11.872Z)

The history recounting in the doc edit faithfully matches the original master text (the `fname.replace(".rallies.csv","")` derivation, the `video_by_stem.get(name)` lookup against bare stems, the "every clip skipped ? `<2 videos`" abort). The "synth train + all eval were unaffected ? they don't join videos" was already in the original blocker text, and blocker #2 confirms eval is local-FS only (separate path), so it doesn't go through this video-join.

Let me do a final precise verification of the exact test function names and the parametrized case count, and confirm the doc's claim that the helper strips "both the `.rallies.csv`/`.csv` suffix AND a leading `^\d{4}-\d{2}-\d{2}_` prefix" matches the regex exactly.

18. user (2026-06-22T12:35:14.447Z)

```
=== exact regex + suffix strip in code ===
271:_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
288:    name = fname.replace(".rallies.csv", "").replace(".csv", "")
289:    return _LABEL_DATE_PREFIX.sub("", name)

=== test function names in tests/test_worker.py ===
17:@pytest.mark.parametrize("raw,expected", [
20:def test_coerce(raw, expected):
24:def test_apply_overrides_on_typed_config():
33:def test_apply_overrides_rejects_bad_format():
39:def test_write_intervals_csv(tmp_path):
50:def test_write_report_json(tmp_path):
58:def test_parser_infer_accepts_set_and_uris():
111:def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
143:def test_setup_logging_levels():
151:def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
178:def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
194:@pytest.mark.parametrize("fname,expected", [
202:def test_label_clip_name_strips_date_prefix(fname, expected):
207:def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
254:def test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
298:def test_cmd_train_detector_skips_unprobeable_and_trajless_videos(tmp_path, monkeypatch):
355:def test_cmd_train_detector_aborts_clean_when_fewer_than_two_videos_join(tmp_path,
monkeypatch):
384:def test_cmd_train_orchestration(tmp_path, monkeypatch):
422:def test_parser_train_detector_args():
429:def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
476:def test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(tmp_path,
monkeypatch):
495:def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
```

19. assistant (2026-06-22T12:35:17.527Z)

All claims verified. Let me run the actual test suite for the worker tests to confirm they pass (verified state, not just present), as the working-agreement requires reproducing claims rather than trusting them.

20. user (2026-06-22T12:35:20.033Z)

```
WORKTREE READY
```

21. user (2026-06-22T12:35:26.627Z)

```
.............................                                            [100%]
29 passed in 2.53s
```

22. assistant (2026-06-22T12:35:31.160Z)

All 29 worker tests pass, including the parametrized helper (6 cases), the join regression, and the skip-branch tests. Let me confirm the parametrized helper test specifically covers the both-suffix claim and clean up the worktree.

23. user (2026-06-22T12:35:36.030Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\AppData\Local\Temp\pr319-wt
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 8 items

tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.rallies.csv-
GX020094] PASSED [ 12%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094.rallies.csv-GX020094]
PASSED [ 25%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094_proxy.rallies.csv-
GX020094_proxy] PASSED [ 37%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-
21_mahadevpura_1.rallies.csv-mahadevpura_1] PASSED [ 50%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[v1.csv-v1] PASSED [ 62%]
tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.csv-GX020094]
PASSED [ 75%]
tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos PASSED [ 87%]
tests/test_worker.py::test_cmd_train_detector_skips_label_with_no_matching_video PASSED [100%]

============================== 8 passed in 1.86s ==============================
error: failed to delete 'C:/Users/avidu/AppData/Local/Temp/pr319-wt': Permission denied
CLEANED
```

24. assistant (2026-06-22T12:35:40.490Z)

All three doc-cited tests pass. The parametrized helper test explicitly covers both `.rallies.csv` and `.csv` suffixes plus the dated and bare cases. Let me clean up the worktree reference.

25. user (2026-06-22T12:35:41.048Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer
62bcdf4 [docs/data-gcs-train-blocker-resolved]
C:/Users/avidu/AppData/Local/Temp/wt-fix
56c1415 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/wt-master
48f9d20 (detached HEAD)
C:/Users/avidu/Projects/_pr319_review
56c1415 (detached HEAD)
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed
7e0881a [perf/wasb-fast-frames]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy
5cb7bbf [bhi-execpolicy]
```

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17
8736e2d [run/boxhill17-highlights]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gap-doc
f04f3f3 [docs/rally-gap-closure]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gate-a-reground
48f9d20 [fix/gate-a-litmus-reground-n15]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hardchop
614325d [feat/w1-hard-cap-fallback]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-stitch
a6adcc5 [fix/runner-stitch-watermark]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/recdoc
9fd4215 [docs/recording-single-match]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/resweep
fe34013 [docs/lovo-resweep-n13]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing
a3b209a [master]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard
5ba6955 [feat/w2-never-empty-safeguard]

26. assistant (2026-06-22T12:35:53.904Z)

The temp worktree directory will be cleaned by prune once the lock releases; the registered worktrees are fine. I have all the evidence I need.

Every factual claim in the doc edit checks out against the actual code and tests in PR #319:

1. **Helper `_label_clip_name()` exists, named exactly that** ? confirmed at `backend/worker/__main__.py:274`.
2. **Strips both `.rallies.csv`/`.csv` suffix AND leading `^\d{4}-\d{2}-\d{2}_` prefix** ? confirmed at lines 271/288-289: `name = fname.replace(".rallies.csv","").replace(".csv","")` then `_LABEL_DATE_PREFIX.sub("", name)` where `_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")`.
3. **Dated label + legacy bare label both resolve to bare `videos/<name>` stem** ? confirmed in the docstring examples and the passing parametrized test.
4. **The three cited tests all exist and pass:**
   - "parametrized helper cases" ? `test_label_clip_name_strips_date_prefix` (6 params, all pass)
   - "dated-label?bare-video cmd_train join regression" ? `test_cmd_train_detector_joins_dated_labels_to_bare_videos` (passes)
   - "per-label skip-branch coverage" ? `test_cmd_train_detector_skips_label_with_no_matching_video` (passes)
5. **PR number/link #319 correct** ? `gh pr view 319` confirms branch `fix/worker-train-dated-label-join`; #321 is the doc PR. §8 tracker row (line 239) matches the body text.
6. **"synth train + all eval were unaffected"** ? confirmed: the synth branch (`else:` at line 431) never touches `video_by_stem`; eval is a separate local-FS-only path (blocker #2).
7. **History recounting faithful** ? matches the original master text verbatim in substance.

I ran the full worker suite (29 passed) and the three cited tests verbosely (8 passed) to verify, not just trust.

One minor note (a nit, not a defect): the prose says the helper "strips both the `.rallies.csv`/`.csv` suffix AND a leading prefix" ? technically the code chains two `.replace()` calls (`.rallies.csv` then `.csv`), which is fine and the test covers both, so the doc's phrasing is accurate.

27. user (2026-06-22T12:36:07.074Z)

Structured output provided successfully

28. assistant (2026-06-22T12:36:10.674Z)

The doc edit in PR #321 is factually accurate. Every claim was verified against the actual code and tests in PR #319, and I reproduced the test results locally (29 worker tests pass, including the 3 specifically cited). No blocking issues ? `ok=true`. Two optional nits flagged for clarity

only (the chained-`.replace()` phrasing and the line-number-free file reference), neither of which is a fabrication or overstatement.